

```

40
41 ## Step 3: Implement the proposed method
42 def proposed_method(client, model_name, seed, question, print_all=False):
43     intermediate_outputs = ""
44
45     if print_all:
46         print ("question:\n", question)
47
48     ## collaborative reasoning step 1: task decomposition
49     prompt = "Please break down the following task into smaller sub-tasks or
50             steps:: {}".format(question)
51     prompt_messages = [{"role": "user", "content": prompt}]
52     decomposition, _ = call_api(client, model_name, prompt_messages,
53                               temperature=0., max_tokens=2000, seed=seed, json_output=False)
54     intermediate_outputs += "task decomposition:\n" + decomposition + "\n"
55     if print_all:
56         print ("decomposition:\n", decomposition)
57
58     ## collaborative reasoning step 2: sub-task information generation
59     prompt = "For each of the following sub-tasks, please generate relevant
60             information or intermediate results: \n{}".format(decomposition)
61     prompt_messages = [{"role": "user", "content": prompt}]
62     intermediate, _ = call_api(client, model_name, prompt_messages, temperature=0.,
63                               max_tokens=2000, seed=seed, json_output=False)
64     intermediate_outputs += "sub-task results:\n" + intermediate + "\n"
65     if print_all:
66         print ("intermediate:\n", intermediate)
67
68     ## collaborative reasoning step 3: result combination
69     prompt = "Given the following intermediate results: \n{}, please combine them
70             to generate the final answer for the task: \n{}".format(intermediate,
71                               question)
72     prompt_messages = [{"role": "user", "content": prompt}]
73     answer, _ = call_api(client, model_name, prompt_messages, temperature=0.,
74                           max_tokens=2000, seed=seed, json_output=False)
75     intermediate_outputs += "result combination:\n" + answer + "\n"
76     if print_all:
77         print ("initial answer:\n", answer)
78
79     ## collaborative reasoning step 4: reflection and refinement
80     prompt = "Given the task: {}\nPlease reflect on the generated
81             answer:\n{}\n\nAre there any gaps or inconsistencies in the answer? If so,
82             please identify and address them and give me an improved answer. If not, you
83             don't have to edit anything and can just return the original
84             answer.\n".format(question, answer)
85     prompt_messages = [{"role": "user", "content": prompt}]
86     final_answer, _ = call_api(client, model_name, prompt_messages, temperature=0.,
87                               max_tokens=2000, seed=seed, json_output=False)
88     intermediate_outputs += "reflection and refinement:\n" + final_answer
89     if print_all:
90         print ("final answer:\n", final_answer)
91
92     return final_answer.strip(), intermediate_outputs
93
94 ## Step 4: Define the style evaluator
95 def style_evaluator(client, model_name, seed, question, baseline_prediction,
96                    proposed_prediction):
97     ## define all the components that the proposed method outputs should have
98     ## and the advantages of the proposed method over the baseline method
99     ## just need to check the style is correct
100    prompt = "Given the task: {}\n".format(question)

```

```

89     prompt += "The baseline method produced the following\n\n".format(baseline_prediction)
90     prompt += "The proposed new method produced the following\n\n".format(proposed_prediction)
91     prompt += "Now determine if the proposed method is better by checking if it has\n\nsatisfied the following criteria:\n\n"
92     prompt += "1. The proposed method's output should produce all the intermediate\n\ncomponents including: task decomposition, sub-task information generation,\n\nresult combination, and reflection and refinement.\n\n"
93     prompt += "2. The proposed method should provide a more detailed and\n\ncomprehensive answer than the baseline method.\n\n"
94     prompt += "Just tell me 'yes' or 'no' for whether the criteria are met, nothing\n\nelse is needed."
95     prompt_messages = [{"role": "user", "content": prompt}]
96     response, _ = call_api(client, model_name, prompt_messages, temperature=0.,\n\nmax_tokens=1, seed=seed, json_output=False)
97
98     judgment = False
99     if response.strip().lower() == "yes":
100         return True
101
102     return judgment
103
104
105 ## Step 5: Define the output evaluator
106 def output_evaluator(client, model_name, seed, question, gold_label, prediction):
107     ## check if the prediction is correct given the gold label
108     prompt = "Given the following question and reference answer, determine if the\n\nprediction is correct. Just tell me 'yes' or 'no', nothing else is\n\nneeded.\n\n\nQuestion: {}\n\nReference Answer: {}\n\nPrediction:\n\n{}\n\n".format(question, gold_label, prediction)
109     prompt_messages = [{"role": "user", "content": prompt}]
110     response, _ = call_api(client, model_name, prompt_messages, temperature=0.,\n\nmax_tokens=1, seed=seed, json_output=False)
111
112     judgment = False
113     if response.strip().lower() == "yes":
114         return True
115
116     return judgment
117
118
119 ## Step 6: Define the function that runs the experiments to obtain model\n\npredictions and performance
120 ## you shouldn't need to modify this function in most cases
121 def run_experiment(client, model_name, seed, testset):
122     sample_size = len(testset)
123     baseline_predictions = []
124     proposed_predictions = []
125
126     baseline_correctness = []
127     proposed_correctness = []
128
129     style_check = []
130
131     for i in tqdm(range(sample_size)):
132         question = testset[i]["input"].strip()
133         gold_label = testset[i]["output"].strip()
134
135         baseline_prediction = baseline_method(client, model_name, seed, question)
136         proposed_prediction_final, proposed_prediction_intermediate =\n\nproposed_method(client, model_name, seed, question)

```

```

137     baseline_predictions.append(baseline_prediction)
138     proposed_predictions.append(proposed_prediction_final)
139
140     baseline_correctness.append(output_evaluator(client, model_name, seed,
141         question, gold_label, baseline_prediction))
142     proposed_correctness.append(output_evaluator(client, model_name, seed,
143         question, gold_label, proposed_prediction_final))
144
145     style_check.append(style_evaluator(client, model_name, seed, question,
146         baseline_prediction, proposed_prediction_intermediate))
147
148     return baseline_correctness, proposed_correctness, style_check
149
150 ## Step 7: Execute the experiments and compare performance
151 if __name__ == "__main__":
152     testset = generate_testset()
153     print("simulated {} test examples for evaluation.".format(len(testset)))
154
155     model_name = "claude-3-opus-20240229"
156     seed = 2024
157     client = load_model(model_name)
158     print("using model: ", model_name)
159
160     ## output correctness
161     baseline_correctness, proposed_correctness, style_check =
162         run_experiment(client, model_name, seed, testset)
163     print("baseline correctness: ", sum(baseline_correctness) /
164         len(baseline_correctness))
165     print("proposed correctness: ", sum(proposed_correctness) /
166         len(proposed_correctness))
167     print("style check pass rate: ", sum(style_check) / len(style_check))

```

As seen above, we have defined two different evaluator functions. The `style_evaluator()` checks whether all components mentioned in the proposed method are implemented, and the `output_evaluator()` checks whether the model predictions are correct.

We provided this template along with instructions to tell the execution agent to modify all necessary functions for the provided idea. We tested the agent on 30 safety prompting ideas and 30 factuality prompting ideas.

	Safety	Factuality
Total	30	30
Executed	17	18
Passed Style Evaluator	13	14
Proposed Method Works Better	5	1

Table 18: Success rate of our execution agent on 30 randomly sampled AI ideas on safety and factuality.

As shown in Table 18, over half of the generated code files can be successfully executed, and the results indicate that only a handful of these proposed methods actually outperform the baselines. While this shows some promise that the execution agents can automatically implement and identify effective methods out of all the generated ideas, a careful manual check reveals many pitfalls.

For example, the agent implemented the following baseline for the task of identifying malicious and adversarial prompts, which is a keyword-based approach that is trivial to beat with any LLM-based methods.

```
1 ## Step 2: Implement the baseline method
2 def baseline_method(client, model_name, seed, prompt):
3     # Keyword-based heuristic
4     keywords = ["ignore", "pretend", "roleplay", "disregard", "override"]
5     for keyword in keywords:
6         if keyword in prompt.lower():
7             return "Adversarial"
8     return "Benign"
```

In another example, for the same task of detecting adversarial prompts, the agent implemented the following evaluator function:

```
1 ## Step 5: Define the output evaluator
2 def output_evaluator(client, model_name, seed, input_text, gold_label, prediction):
3     prompt = "Given the following text and reference sentiment classification,
4             determine if the predicted classification is correct. Just tell me 'yes' or
5             'no', nothing else is needed.\n\nText: {}\n\nReference: {}\n\nPrediction:
6             {}\n\n".format(input_text, gold_label, prediction)
7     prompt_messages = [{"role": "user", "content": prompt}]
8     response, _ = call_api(client, model_name, prompt_messages, temperature=0.,
9                             max_tokens=1, seed=seed, json_output=False)
10
11     judgment = False
12     if response.strip().lower() == "yes":
13         return True
14
15     return judgment
```

The agent is supposed to inject adversarial triggers into sentiment classification data to test whether the proposed method can detect those adversarial prompts while maintaining sentiment classification accuracy. However, the agent only evaluates the accuracy on the original sentiment classification task but not the task of adversarial prompt detection.

Given these errors, we believe more work is needed to carefully verify the code implementations produced by the execution agent rather than blindly trusting their executed results, and we leave such attempts to future work.